

 c0a25062d3 / ProjExD_Group14  | 

 Code  Pull requests  Agents  Actions  Projects  Wiki  Security and quality  Insights

ProjExD_Group14 / koukaton_up.py 

 c0a25062d3 機能追加：当たり判定

7887599 · 28 minutes ago 

293 lines (234 loc) · 10.2 KB

```
1  import pygame
2  import sys
3  import os
4  import random
5
6  # カレントディレクトリの固定
7  os.chdir(os.path.dirname(os.path.abspath(__file__)))
8
9  WIDTH, HEIGHT = 600, 800
10 FPS = 60
11 GRAVITY = 0.6
12
13 BLACK = (20, 20, 25)
14 BLUE = (100, 150, 255)
15 GREEN = (120, 120, 120)
16 RED = (255, 100, 100)
17 WHITE = (255, 255, 255)
18 GOLD = (255, 215, 0) # ゴールブロック用の金色
19
20 # =====
21 # 1. データ構造（クラス）の定義
22 # =====
23
24 class Player:
25     def __init__(self):
26         self._image = pygame.Surface((30, 40))
27         self._image.fill(BLUE)
28         self._rect = self._image.get_rect(center=(WIDTH // 2, HEIGHT - 100))
29
30         self._vel_x = 0
31         self._vel_y = 0
32         self._on_ground = False
33
34         self._is_charging = False
35         self._charge_power = 0
36         self._max_charge = 20.0
37         self._direction = 1
38
39         self._is_clear = False # ゴールしたかどうかのフラグ
40
41     # --- ゲッター ---
42     def get_image(self): return self._image
43     def get_rect(self): return self._rect
```

```
44     def get_vel_x(self): return self._vel_x
45     def get_vel_y(self): return self._vel_y
46     def get_on_ground(self): return self._on_ground
47     def get_is_charging(self): return self._is_charging
48     def get_charge_power(self): return self._charge_power
49     def get_max_charge(self): return self._max_charge
50     def get_direction(self): return self._direction
51     def get_is_clear(self): return self._is_clear
52
53     # --- セッター ---
54     def set_vel_x(self, value): self._vel_x = value
55     def set_vel_y(self, value): self._vel_y = value
56     def set_on_ground(self, value): self._on_ground = value
57     def set_is_charging(self, value): self._is_charging = value
58     def set_charge_power(self, value): self._charge_power = value
59     def set_direction(self, value): self._direction = value
60     def set_is_clear(self, value): self._is_clear = value
61
62
63     class Platform:
64         # 足場ごとに色を変えられるように引数追加
65         def __init__(self, x, y, w, h, color):
66             self._image = pygame.Surface((w, h))
67             self._image.fill(color)
68             self._rect = self._image.get_rect(topleft=(x, y))
69
70         def get_image(self): return self._image
71         def get_rect(self): return self._rect
72
73
74         # =====
75         # 2. クラスを操作する関数
76         # =====
77
78     def update_player(player, keys, platforms, goal_block):
79         """プレイヤーの物理挙動と状態を更新する関数"""
80
81         # ゴール済みの場合は入力を受け付けない（重力で落ちるだけ）
82         if not player.get_is_clear():
83             # 1. 地面にいる時の処理
84             if player.get_on_ground():
85                 player.set_vel_x(0)
86
87                 if keys[pygame.K_LEFT] or keys[pygame.K_a]:
88                     player.set_direction(-1)
89                 if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
90                     player.set_direction(1)
91
92                 if keys[pygame.K_SPACE]:
93                     player.set_is_charging(True)
94                     current_power = player.get_charge_power()
95                     if current_power < player.get_max_charge():
96                         player.set_charge_power(current_power + 0.4)
97                 else:
98                     if player.get_is_charging():
99                         power = player.get_charge_power()
100                         direction = player.get_direction()
101
102                         player.set_vel_y(-power * 0.9 - 5)
```

```
103         player.set_vel_x(direction * (power * 0.4 + 2))
104
105         player.set_is_charging(False)
106         player.set_charge_power(0)
107         player.set_on_ground(False)
108
109     # 2. 重力の適用
110     if not player.get_on_ground():
111         player.set_vel_y(player.get_vel_y() + GRAVITY)
112
113     rect = player.get_rect()
114
115     # x軸方向の移動と、足場の【左右】の当たり判定
116     rect.x += player.get_vel_x()
117     if rect.left < 0:
118         rect.left = 0
119         player.set_vel_x(player.get_vel_x() * -0.5)
120         player.set_direction(1)
121     if rect.right > WIDTH:
122         rect.right = WIDTH
123         player.set_vel_x(player.get_vel_x() * -0.5)
124         player.set_direction(-1)
125
126     # 追加：足場の左右側面との衝突をチェック
127     for plat in platforms:
128         plat_rect = plat.get_rect()
129         if rect.colliderect(plat_rect):
130             if player.get_vel_x() > 0: # 右移動中に衝突
131                 rect.right = plat_rect.left
132                 player.set_vel_x(player.get_vel_x() * -0.5)
133                 player.set_direction(-1)
134             elif player.get_vel_x() < 0: # 左移動中に衝突
135                 rect.left = plat_rect.right
136                 player.set_vel_x(player.get_vel_x() * -0.5)
137                 player.set_direction(1)
138
139     # Y軸方向の移動
140     rect.y += int(player.get_vel_y())
141
142     # --- 3. ゴール（天井）との当たり判定 ---
143     goal_rect = goal_block.get_rect()
144     if rect.colliderect(goal_rect):
145         # 上昇中に天井の底にぶつかったらゴール
146         if player.get_vel_y() < 0:
147             rect.top = goal_rect.bottom
148             player.set_vel_y(0)
149             player.set_is_clear(True)
150
151     # --- 4. 足場との当たり判定 ---
152     player.set_on_ground(False)
153
154     for plat in platforms:
155         plat_rect = plat.get_rect()
156         if rect.colliderect(plat_rect):
157             # もともとの上面判定（落下中の着地判定）
158             if player.get_vel_y() > 0:
159                 if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:
160                     rect.bottom = plat_rect.top
161                     player.set_vel_y(0)
```

```
162         player.set_vel_x(0)
163         player.set_on_ground(True)
164         break
165     # 追加：上昇中に足場の【下側（天井部分）】に頭をぶつけた判定
166     elif player.get_vel_y() < 0:
167         rect.top = plat_rect.bottom
168         player.set_vel_y(0) # 上昇を止める
169         break
170
171
172 # =====
173 # 3. メインループ
174 # =====
175
176 ✓ def main():
177     pygame.init()
178     screen = pygame.display.set_mode((WIDTH, HEIGHT))
179     pygame.display.set_caption("Jump King - 10 Floors to Goal")
180     clock = pygame.time.Clock()
181     font = pygame.font.SysFont(None, 36)
182     large_font = pygame.font.SysFont(None, 72)
183
184     player = Player()
185
186     # --- 10層分のマップ生成 ---
187     platforms = []
188     platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, GREEN)) # スタート床
189
190     TOTAL_FLOORS = 10
191     # ゴール（天井）のY座標：スタート床から 10層分(800px * 10) 上へ
192     goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)
193
194     # 金色の天井ブロックを生成
195     goal_block = Platform(0, goal_y, WIDTH, 40, GOLD)
196
197     # ゴールに到達するまで足場を生成し続ける
198     platform_y = HEIGHT - 150
199     while platform_y > goal_y + 100:
200         w = random.randint(60, 130)
201         x = random.randint(0, WIDTH - w)
202         platforms.append(Platform(x, platform_y, w, 20, GREEN))
203         gap_y = random.randint(80, 130)
204         platform_y -= gap_y
205
206     camera_y = 0
207     max_height = 0
208
209     running = True
210     while running:
211         for event in pygame.event.get():
212             if event.type == pygame.QUIT:
213                 running = False
214
215         keys = pygame.key.get_pressed()
216
217         # 関数呼び出し（ゴールブロックも渡して判定させる）
218         update_player(player, keys, platforms, goal_block)
219
220         p_rect = player.get_rect()
```

```
221
222 # --- カメラ処理 ---
223 SCROLL_TOP_MARGIN = 200
224 if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
225     camera_y = p_rect.y - SCROLL_TOP_MARGIN
226
227 SCROLL_BOTTOM_MARGIN = 150
228 camera_easing_down = 0.08
229 if p_rect.bottom - camera_y > HEIGHT - SCROLL_BOTTOM_MARGIN:
230     target_camera_y = p_rect.bottom - (HEIGHT - SCROLL_BOTTOM_MARGIN)
231     camera_y += (target_camera_y - camera_y) * camera_easing_down
232
233 if camera_y > 0:
234     camera_y = 0
235
236 # --- スコア・階層計算 ---
237 current_height = (HEIGHT - 40) - p_rect.bottom
238 if current_height > max_height:
239     max_height = current_height
240
241 # 今いる階層 (800px を 1階 とする)
242 current_floor = min((current_height // HEIGHT) + 1, TOTAL_FLOORS)
243
244 # --- 描画処理 ---
245 screen.fill(BLACK)
246
247 # ゴールの描画
248 goal_rect = goal_block.get_rect()
249 goal_draw_y = goal_rect.y - camera_y
250 if -100 < goal_draw_y < HEIGHT + 100:
251     screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))
252
253 # 足場の描画
254 for plat in platforms:
255     plat_rect = plat.get_rect()
256     draw_y = plat_rect.y - camera_y
257     if -50 < draw_y < HEIGHT + 50:
258         screen.blit(plat.get_image(), (plat_rect.x, draw_y))
259
260 # プレイヤーの描画
261 player_draw_y = p_rect.y - camera_y
262 screen.blit(player.get_image(), (p_rect.x, player_draw_y))
263
264 # チャージゲージと方向指示器
265 if player.get_is_charging() and not player.get_is_clear():
266     gauge_width = (player.get_charge_power() / player.get_max_charge()) * 40
267     pygame.draw.rect(screen, RED, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
268
269 arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
270 pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))
271
272 # 情報表示 (UI)
273 ui_text = font.render(f"Floor: {current_floor} / {TOTAL_FLOORS}    Max: {max_height // 10}m", True, WHITE)
274 screen.blit(ui_text, (10, 10))
275
276 # ゴールした時の演出
277 if player.get_is_clear():
278     clear_text = large_font.render("CLEAR!!", True, GOLD)
279     text_rect = clear_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 50))
```

```
280         screen.blit(clear_text, text_rect)
281
282         sub_text = font.render("CONGRATULATIONS!", True, WHITE)
283         sub_rect = sub_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 + 10))
284         screen.blit(sub_text, sub_rect)
285
286         pygame.display.flip()
287         clock.tick(FPS)
288
289         pygame.quit()
290         sys.exit()
291
292     if __name__ == "__main__":
293         main()
```